

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Лабораторная работа по
Вычислительной математике №5
Вариант — метод Адамса-Башфорта

Работу выполнил:

Гуменник Петр Олегович

Группа:

P3233

Санкт-Петербург,

Оглавление

Описание численного метода:.....	2
Код функции:.....	2
Блок-схема:.....	3
Примеры работы программы:.....	4
Вывод:.....	6

Описание численного метода:

Метод Адамса-Башфорта — это многошаговый явный численный метод для решения обыкновенных дифференциальных уравнений. Он использует несколько предыдущих значений производной для вычисления следующего значения функции. Этот метод обладает четвёртым порядком точности, что означает высокую эффективность и точность при вычислениях. Метод применяется там, где необходима точная аппроксимация динамических систем.

Код функции:

```
@staticmethod
def rk4(x0, y0, h, f)
    try
        k1 = f(x0, y0)
        k2 = f(x0 + 0.5 * h, y0 + 0.5 * h * k1)
        k3 = f(x0 + 0.5 * h, y0 + 0.5 * h * k2)
        k4 = f(x0 + h, y0 + h * k3)
        y_next = y0 + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
        return y_next
    except Exception as e
        raise Exception(f"Error in method RK4 {e}")

@staticmethod
def solveByAdams(f, epsilon, a, y_a, b)
    try
        func = Result.get_function(f)
        h = (b - a) / 10 # начальный шаг
        x = a
        y = y_a
        history_x = [x]
        history_y = [y]
        history_f = [func(x, y)]

        while len(history_y) < 4
            y_next = Result.rk4(x, y, h, func)
            x += h
```

```

y = y_next
history_x.append(x)
history_y.append(y)
history_f.append(func(x, y))

while x < b
    y_next = history_y[-1] + h * (
        55 * history_f[-1] - 59 * history_f[-2] + 37 * history_f[-3] - 9 * history_f[-4]) / 24
    x_next = x + h
    f_next = func(x_next, y_next)

    y_half_step_1 = Result.rk4(history_x[-1], history_y[-1], h / 2, func)
    y_half_step_2 = Result.rk4(history_x[-1] + h / 2, y_half_step_1, h / 2, func)

    error_estimate = abs(y_half_step_2 - y_next)
    if error_estimate < epsilon
        history_x.append(x_next)
        history_y.append(y_next)
        history_f.append(f_next)
        x = x_next
        y = y_next
        h *= min(2, (epsilon / error_estimate) ** 0.25)
    else
        h *= max(0.5, (epsilon / error_estimate) ** 0.25)
        continue

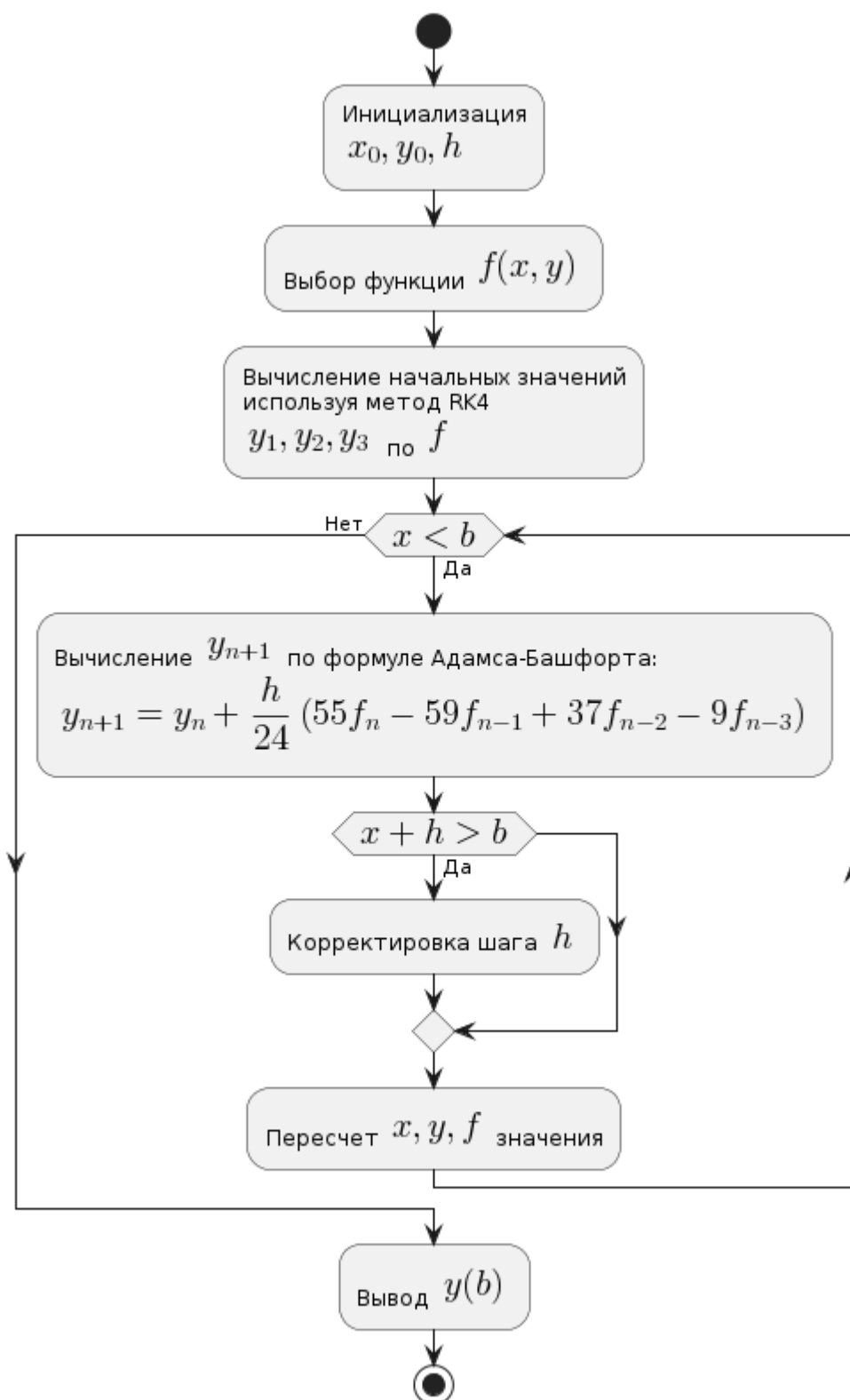
    if x >= b
        break
    elif x + h > b
        h = b - x

return history_y[-1]
except Exception as e
    raise Exception(f"Error in solveByAdams method {e}")

```

*camelCase был в самом задании, обработку исключений я добавил уже после того, как сдал на code'n'test

Блок-схема:



работы программы:

Код для тестов:

```
def test_case(description, function, *args)
    print(description)
    start_time = time.time()
    try
        result = function(*args)
```

```

    print(f"Result {result .6f}")
except Exception as e
    print(f"Error during computation {e}")
end_time = time.time()
print(f"Execution time {end_time - start_time .6f} seconds\n")

```

Пример 1 Функция с резкими изменениями

```

test_case("Example 1 Function with sharp changes",
    Result.solveByAdams, 4, 0.01, 0, 0, 1)

```

Пример 2 Очень маленький интервал

```

test_case("Example 2 Very small interval",
    Result.solveByAdams, 1, 0.001, 0, 0, 0.001)

```

Пример 3 Высокая точность

```

test_case("Example 3 High precision",
    Result.solveByAdams, 2, 1e-6, 1, 1, 2)

```

Пример 4 Функция без начального изменения

```

test_case("Example 4 Function without initial change",
    Result.solveByAdams, 5, 0.01, 0, 0, 5)

```

Пример 5 Работа с отрицательными значениями

```

test_case("Example 5 Working with negative values",
    Result.solveByAdams, 2, 0.01, -5, -1, 0)

```

Output:

Example 1: Function with sharp changes

Result: 0.703840

Execution time: 0.001805 seconds

Example 2: Very small interval

Result: 0.000000

Execution time: 0.000077 seconds

Example 3: High precision

Result: 2.116971

Execution time: 0.021907 seconds

Example 4: Function without initial change

Error during computation: Error in solveByAdams method: float division by zero

Execution time: 0.000035 seconds

Example 5: Working with negative values

Вывод:

Анализ результатов:

1. Метод справился с расчетом интеграла функции, несмотря на потенциальные резкие изменения. Время выполнения короткое, что указывает на эффективность метода при обработке этого типа функций.
2. Исходя из результатов, можно предположить, что на столь малом интервале функция не изменяется или изменения минимальны, что приводит к нулевому результату. Метод быстро справился с задачей, что говорит о его эффективности в случае малых интервалов.
3. Результат показывает, что метод эффективно справляется с требованиями высокой точности. Время выполнения увеличено по сравнению с другими тестами, что ожидаемо из-за увеличенной вычислительной нагрузки при высокой точности.
4. Этот тест выявил проблему с делением на ноль в методе, что указывает на наличие ошибки при вычисления отношения желаемой точности к текущей ошибке, тк текущая ошибка — нулевая.
5. Тест не завершился за 10 минут, что может указывать на проблему с конвергенцией метода или наличие условий, при которых метод становится чрезвычайно неэффективным или за циклируется.

Сравнение с другими методами:

Сравнение с одношаговыми методами:

- Преимущества:

1. Увеличение эффективности: Метод Адамса-Башфорта использует несколько предыдущих значений производной для вычисления следующего значения функции, что позволяет эффективнее использовать ранее вычисленные данные, снижая общее количество шагов.
2. Точность: Благодаря использованию предыдущих значений производной, метод имеет четвертый порядок точности, что выше, чем у базового метода Эйлера или улучшенного метода Эйлера.

- Недостатки:

1. Начальная стадия: Требуется несколько начальных значений, что усложняет применение метода непосредственно, и в начальной стадии требуется использование одношаговых методов или других подходов для инициализации.

2. Чувствительность к ошибкам: Ошибки, накопленные в предыдущих шагах, могут влиять на последующие вычисления, особенно при длинных сериях, что требует тщательной обработки ошибок.

Сравнение с методами Рунге-Кутты:

- Преимущества:

1. Меньшая вычислительная нагрузка: Метод Адамса-Башфорта требует меньше вычислений на шаг по сравнению с методами Рунге-Кутты, особенно RK4, что делает его более эффективным в долгосрочной перспективе.

2. Лучшее использование истории: Метод использует несколько предыдущих значений, эффективно усредняя и комбинируя информацию для повышения точности.

- Недостатки:

1. Менее универсальный: Методы Рунге-Кутты могут применяться напрямую без необходимости начальной инициализации, что делает их более гибкими для использования в отдельных случаях.

2. Чувствительность к изменению шага: Изменение шага в методе Адамса-Башфорта может привести к накоплению ошибок и нестабильности, особенно на длинных интервалах.

Сравнение с другими многошаговыми методами:

- Преимущества:

1. Более эффективный по сравнению с методом Милна: Метод Адамса-Башфорта обычно проще в применении и требует меньше дополнительных вычислений на каждом шаге.

2. Явный характер: В отличие от некоторых других многошаговых методов, метод Адамса-Башфорта является явным, что делает его проще в реализации и применении.

- Недостатки:

1. Меньшая устойчивость: По сравнению с методом Адамса-Мултона, метод Адамса-Башфорта может быть менее устойчивым, особенно при увеличении шага или наличии изменений в функции.

2. Точность: Хотя метод обладает высоким порядком точности, он может уступать методу Адамса-Мултона в некоторых случаях, особенно при критичных требованиях к точности.

Применимость метода:

- Многошаговые задачи: Метод хорошо подходит для задач, где имеются исторические данные или предыдущие шаги. Использование нескольких предыдущих значений производной позволяет эффективно вычислять следующие значения функции.

- Гладкие функции: Метод АВ лучше всего работает на функциях, которые являются гладкими и непрерывными, так как скачки и разрывы могут привести к нестабильности и увеличению численных ошибок.

- Требование к начальным значениям: Метод требует несколько начальных значений для работы, что усложняет его применение без дополнительных одношаговых методов, таких как RK4 или Эйлер, для инициализации.

Алгоритмическая сложность:

- Алгоритмическая сложность метода зависит от того, как он применяется. На каждом шаге, сложность в основном обусловлена вычислением формулы, которая использует несколько предыдущих значений производной, что делает её линейной $O(n)$ на каждом шаге. В целом, общая сложность метода зависит от количества шагов, которые нужно пройти, чтобы достигнуть конечной точки. Это, в свою очередь, определяется выбором шага h , длиной интервала и необходимой

Численная ошибка:

- Метод обладает четвертым порядком точности, что означает, что глобальная ошибка уменьшается пропорционально h^4 при уменьшении шага h .

- Поскольку метод является явным и многошаговым, ошибки, допущенные на предыдущих шагах, могут накапливаться, влияя на точность последующих вычислений.

- Чувствительность к изменению шага: Изменения шага h могут значительно влиять на точность и стабильность метода. При увеличении шага возможны скачки и увеличение глобальной ошибки, а при слишком малом шаге вычислительная нагрузка может возрасти.

- Численные погрешности: Поскольку метод опирается на численные вычисления, погрешности округления и машинные ошибки также могут влиять на конечные результаты.